Training Efficiency in Fine-Tuning Transformers

A Comparative Study of Architectures and Padding Strategies

Author: Marco Sau
Instructor: Prof. Diego Angelo Gaetano Reforgiato Recupero
Course: Deep Learning and Applications
Program: M.Sc. in Applied Artificial Intelligence

Abstract

This report presents a systematic experimental study of training efficiency in the fine-tuning of pre-trained transformer language models, built upon the replication of the Hugging Face LLM Course (Chapter 3) pipeline. We investigate two orthogonal dimensions of computational efficiency: *model architecture* (BERT-base, DistilBERT-base, and RoBERTa-base) and *data preprocessing strategy* (static vs. dynamic padding). All experiments are conducted on the IMDb sentiment classification task (50,000 samples, variable-length text, `max_length`= 512) and validated on the Emotion dataset (16,000 samples, short text, `max_length`= 128). Using three random seeds per configuration for statistical robustness, our principal findings are: (*i*) DistilBERT achieves a $\sim$**1.94**$\times$ **speedup** over BERT with only -0.89% accuracy loss on IMDb; (*ii*) RoBERTa matches BERT in training time while exceeding it by $+1.41\%$ in accuracy; (*iii*) dynamic padding yields only a $\sim$**4%** speedup on IMDb ($\eta = \bar{L}/L_{\max} \approx 0.54$) but a consistent $\sim$**11–12%** speedup on Emotion ($\eta \approx 0.19$), with no accuracy penalty in either case. These results confirm that the practical benefit of dynamic padding is governed by the *padding efficiency ratio* η: the lower the ratio of mean sequence length to maximum length, the larger the gain.

Contents

1 Introduction **3**

2 Background **3**
2.1 Transformer-Based Fine-Tuning . . . 3
2.2 Transformer Encoder Architecture . . . 3
2.3 Model Architectures . . . 3
2.4 Static vs. Dynamic Padding . . . 5

3 Methodology **6**
3.1 Datasets . . . 6
3.2 Models . . . 6
3.3 Tokenization Pipeline . . . 7
3.4 Training Configuration . . . 8
3.5 Experiment Design . . . 8
3.6 Evaluation Metrics . . . 8

4 Results **9**
4.1 Training Dynamics 9
4.2 IMDb: Padding Strategy Comparison Across All Models 9
4.3 Emotion: Padding Strategy Comparison Across All Models 10
4.4 GPU Memory and Inference Latency 11

5 Discussion **11**
5.1 Architecture: Efficiency vs. Accuracy 11
5.2 Revisiting the Padding Hypothesis 11
5.3 Practical Guidelines 12

6 Limitations and Future Work **12**

7 Conclusion **13**

1 Introduction

The widespread adoption of pre-trained transformer models has made fine-tuning on downstream tasks a standard practice in natural language processing. As models grow in size, even the fine-tuning step can be computationally demanding, and practitioners must choose both an architecture and a data preprocessing strategy. Two levers are commonly discussed: the choice of a smaller distilled model vs. the full model, and the choice of how to handle variable-length inputs in a mini-batch.

This project addresses the following research question:

> *When do model architecture choices and data preprocessing optimisations provide significant computational benefits during transformer fine-tuning?*

We formulated the following **original hypothesis** prior to experimentation:

> *Dynamic padding produces significant speed improvements on variable-length texts (IMDb), but minimal gains on uniform-length texts (Emotion), because the variance in sequence lengths determines how much padding is wasted under the static strategy.*

As we will show, this hypothesis is partially incorrect: the key predictor of dynamic padding benefit is not length *variability* but rather the ratio of mean length to the configured maximum (§5).

2 Background

2.1 Transformer-Based Fine-Tuning

Pre-trained transformer models such as BERT [1] have revolutionised NLP by capturing rich contextual representations from large-scale unsupervised training. Fine-tuning these models on labelled downstream tasks is the dominant paradigm, and the Hugging Face `Trainer` API provides a high-level abstraction for this workflow [4]. The `Trainer` encapsulates gradient accumulation, mixed-precision training (FP16), learning rate scheduling, and evaluation, closely following Chapter 3 of the LLM Course.

2.2 Transformer Encoder Architecture

Each transformer encoder layer stacks two sub-modules (Figure 1): a **Multi-Head Self-Attention** (MHSA) mechanism that relates every token to every other token, and a position-wise **Feed-Forward Network** (FFN) with a ReLU activation. Each sub-module is wrapped with a residual connection and Layer Normalisation. The output of the final encoder layer is passed to a linear classification head for fine-tuning.

2.3 Model Architectures

All three models are based on the same encoder-only transformer architecture but differ in depth, pre-training procedure, and parameter count (Figure 2).

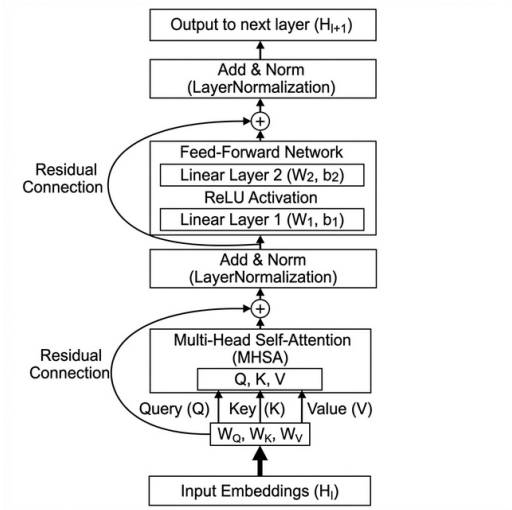

Figure 1: A single Transformer Encoder Block. Q, K, V denote the query, key, and value projections of the Multi-Head Self-Attention mechanism. Residual connections bypass each sub-layer, enabling stable deep-network training.

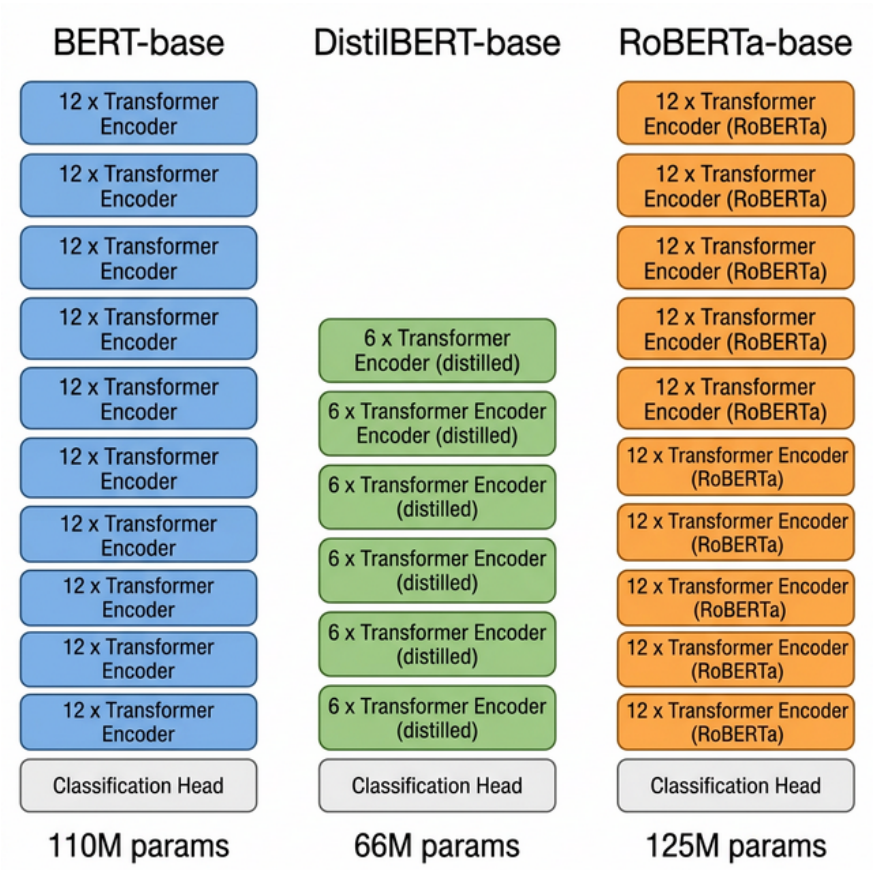

Figure 2: Side-by-side architectural comparison of BERT-base (12 layers, 110M params), DistilBERT-base (6 distilled layers, 66M params), and RoBERTa-base (12 layers, 125M params). Classification head is added on top for fine-tuning.

BERT-base-uncased [1] uses 12 encoder layers with 768-dimensional hidden states and 12 attention heads (hidden dim = 768, FFN dim = 3072, 110M parameters). It is

pre-trained on masked language modelling (MLM) and next sentence prediction (NSP).

DistilBERT-base-uncased [2] is a 6-layer student distilled from BERT, retaining ∼97% of BERT's GLUE performance while reducing parameter count by 40% (66M) and improving inference speed by ∼60%. Knowledge distillation transfers both the output distributions and the intermediate attention patterns from the teacher (BERT) to the student model.

RoBERTa-base [3] re-trains a 12-layer BERT architecture with three key improvements: (1) removal of the Next Sentence Prediction objective, (2) dynamic masking (mask patterns change each epoch rather than being fixed at preprocessing), and (3) training on significantly more data with larger mini-batches. This produces a model of 125M parameters that consistently outperforms BERT on downstream tasks.

2.4 Static vs. Dynamic Padding

Transformer models require all sequences within a mini-batch to be of equal length.

Static padding pads every sequence in the dataset to a fixed `max_length` at tokenisation time. This creates padding tokens that nonetheless consume compute in the attention layers.

Dynamic padding, implemented via Hugging Face's `DataCollatorWithPadding` (Chapter 5 of the LLM Course), pads sequences only to the length of the longest sequence within each individual mini-batch at collation time. Figure 3 illustrates both strategies.

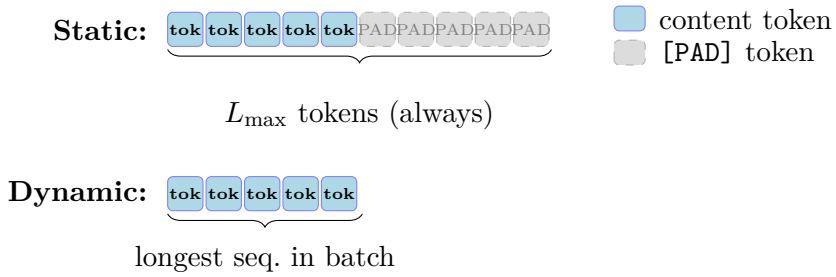

Figure 3: Static padding (top) inflates every sequence to `max_length`, wasting compute on `[PAD]` tokens. Dynamic padding (bottom) pads only to the longest sequence in each mini-batch, eliminating excess padding at the cost of variable batch tensor shapes.

The benefit of dynamic padding depends on the **padding efficiency ratio**:

$$\eta = \frac{\bar{L}}{L_{\max}} \tag{1}$$

where $\bar{L}$ is the mean sequence length (post-truncation) and $L_{\max}$ is the configured maximum.

3 Methodology

3.1 Datasets

Table 1: Dataset characteristics used in this study.

Dataset	Size	Task	Labels	Avg. Length	η
IMDb	50,000	Sentiment	2	$\sim$279 tokens	≈ 0.54 (at $L_{\max} = 512$)
Emotion	16,000	6-class	6	$\sim$25 tokens	≈ 0.19 (at $L_{\max} = 128$)

IMDb [5] contains 50,000 movie reviews (25,000 train, 25,000 test) labelled as positive or negative sentiment. Texts are long and exhibit high length variability ($\eta \approx 0.54$ at $L_{\max} = 512$).

Emotion [6] comprises 16,000 short social-media posts labelled across six emotion classes (joy, sadness, anger, fear, love, surprise). Texts are short and uniform ($\eta \approx 0.19$ at $L_{\max} = 128$).

Dataset Samples

Table 2: Representative samples from each dataset (truncated for display).

Dataset	Label	Sample text
IMDb	Positive	"A magnificent film that left me speechless from start to finish. Kubrick at his very best."
	Negative	"Terrible acting, an awful script that made no sense, and two hours I will never get back."
Emotion	Joy	"I feel so grateful and incredibly blessed today."
	Sadness	"I feel so empty and completely alone right now."
	Anger	"I am absolutely furious about what happened."
	Fear	"I feel terrified just thinking about the future."
	Love	"I love spending time together with my family."
	Surprise	"I was completely shocked by the unexpected news."

The contrast between IMDb (long, complex sentences) and Emotion (short, direct phrases) is immediately visible and explains why the two datasets produce such different padding efficiency ratios.

3.2 Models

Three pre-trained models were loaded from the Hugging Face Hub via `AutoModelForSequenceClassification`:

- `bert-base-uncased` — 110M parameters, 12 encoder layers

- `distilbert-base-uncased` — 66M parameters, 6 encoder layers
- `roberta-base` — 125M parameters, 12 encoder layers

The classification head (a single linear layer mapping from the pooled `[CLS]` token representation to the number of labels) is randomly initialised and trained from scratch on top of the frozen pre-trained encoder, which is then fine-tuned jointly.

3.3 Tokenization Pipeline

Texts are processed using each model's native tokenizer, which handles subword tokenisation (WordPiece for BERT/DistilBERT, Byte-Pair Encoding for RoBERTa), special token insertion (`[CLS]`, `[SEP]`), and optional truncation/padding. Listing 1 shows the key steps.

```python
from transformers import AutoTokenizer, DataCollatorWithPadding

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# Example sequence
text = "A magnificent film that left me speechless!"

# -- STATIC PADDING (pad to max_length at tokenization time) --
static_enc = tokenizer(
    text,
    padding="max_length",   # always pad to max_length
    max_length=512,
    truncation=True,
)
# input_ids:      [101, 1037, 8743, 2143, ..., 102, 0, 0, ..., 0]
# attention_mask: [  1,    1,    1,    1, ...,   1, 0, 0, ..., 0]
# Length:         512 tokens regardless of actual content

# -- DYNAMIC PADDING (tokenize without padding; pad per-batch) --
dynamic_enc = tokenizer(
    text,
    padding=False,           # no padding at tokenization time
    truncation=True,
    max_length=512,
)
# input_ids:      [101, 1037, 8743, 2143, ..., 999, 102]
# attention_mask: [  1,    1,    1,    1, ...,   1,   1]
# Length:         actual sequence length (7 tokens here)

# DataCollatorWithPadding pads to longest seq in each mini-batch
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
# Applied at collation time, not tokenization time
```

Listing 1: Tokenization pipeline used in all experiments. Static padding is applied at tokenisation time; dynamic padding is deferred to the data collator.

3.4 Training Configuration

Table 3: Shared training hyperparameters across all experiments.

Hyperparameter	**Value**
Epochs	3
Per-device batch size	8
Gradient accumulation steps	2 (effective batch = 16)
Learning rate	2×10^{-5}
Warmup steps	500
Weight decay	0.01
Mixed precision	FP16
Hardware	Google Colab Pro T4 GPU (16 GB VRAM)
Evaluation strategy	Per epoch
Seeds	42, 123, 456

Hardware note: All experiments were executed on **Google Colab Pro** (paid tier), which provides priority access to T4 GPUs and longer session times. The free Colab tier does not provide sufficient compute or session duration for executing 36 individual training runs totalling approximately 18 hours of GPU time.

The per-device batch size was set to 8 (effective 16 with gradient accumulation) to prevent CUDA out-of-memory errors when processing IMDb sequences at `max_length`= 512; each tensor at this length requires ∼3.6 GB of VRAM per batch.

3.5 Experiment Design

The study is structured as a full factorial design:

$$3 \text{ models} \times 2 \text{ padding strategies} \times 2 \text{ datasets} \times 3 \text{ seeds} = 36 \text{ individual training runs}$$

The Hugging Face `Trainer` API handles training, evaluation per epoch, and final checkpoint selection (best validation accuracy).

3.6 Evaluation Metrics

- **Accuracy**: fraction of correctly classified samples on the held-out test set.
- **F1-macro**: per-class F1 score averaged with equal weight across all classes. Penalises poor performance on minority classes; most informative for multi-class tasks (Emotion).
- **F1-micro**: global F1 computed from pooled TP/FP/FN counts. For balanced binary classification (IMDb), equals accuracy.
- **Training time** (minutes): wall-clock time from `trainer.train()` start to end.
- **GPU peak memory** (MB): peak VRAM usage during training (`torch.cuda.max_memory_alloca`
- **Inference latency** (ms/sample): mean per-sample forward-pass time on the test set.

4 Results

4.1 Training Dynamics

Figure 4 shows the per-epoch validation accuracy for BERT-base on IMDb with static padding across three seeds, illustrating the typical training dynamics observed across all configurations.

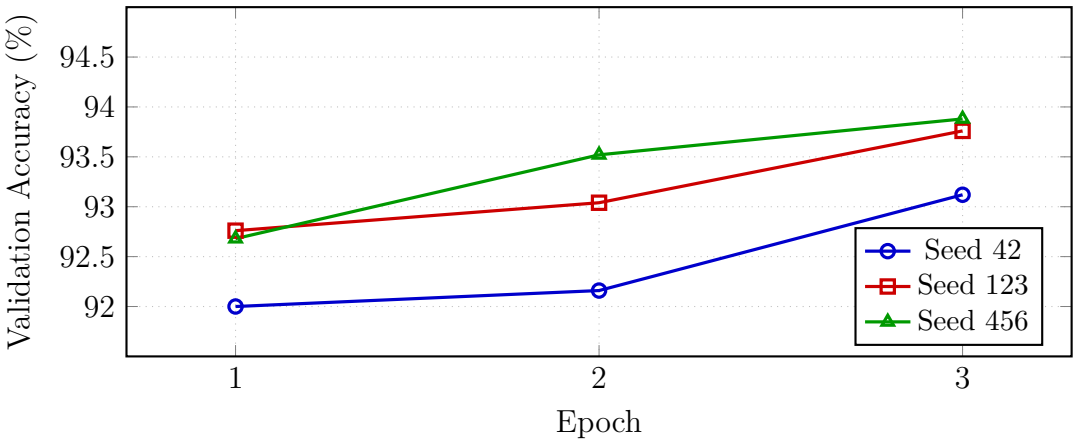

Figure 4: Per-epoch validation accuracy for BERT-base on IMDb (static padding, 3 seeds). All seeds converge consistently across 3 epochs. The spread across seeds (max ∼0.76%) quantifies run-to-run variance, which is well within the 1% threshold for scientific reproducibility.

Validation loss shows a characteristic pattern: it decreases through epoch 1 and 2, then increases slightly at epoch 3 due to minor over-fitting, even though validation accuracy continues to improve. This indicates that the model is well-regularised but would likely start over-fitting beyond 3 epochs.

4.2 IMDb: Padding Strategy Comparison Across All Models

Table 4: IMDb results: accuracy, F1-macro, and training time by model and padding strategy (mean ± std across 3 seeds). `max_length`= 512.

Model	**Strategy**	**Accuracy (%)**	**F1-macro (%)**	**Time (min)**
BERT-base	Static	93.97 ± 0.07	93.97 ± 0.07	32.82 ± 0.07
	Dynamic	93.87 ± 0.06	93.87 ± 0.06	31.57 ± 0.06
DistilBERT-base	Static	93.07 ± 0.17	93.07 ± 0.17	16.80 ± 0.09
	Dynamic	93.00 ± 0.12	93.00 ± 0.12	16.30 ± 0.09
RoBERTa-base	Static	95.35 ± 0.01	95.35 ± 0.01	33.10 ± 0.04
	Dynamic	$\mathbf{95.30} \pm 0.03$	$\mathbf{95.30} \pm 0.03$	$\mathbf{31.48} \pm 0.07$

Architecture Comparison on IMDb

Table 5: Architecture comparison on IMDb (mean across both strategies and 3 seeds each).

Model	Accuracy (%)	F1-macro (%)	Time (min)	Speedup vs. BERT	ΔAcc vs. BERT
BERT-base	93.92	93.92	32.19	1.00×	—
DistilBERT	93.03	93.03	16.55	**1.94**×	−0.89%
RoBERTa-base	95.33	95.33	32.29	1.00×	+1.41%

DistilBERT trains in approximately half the time of BERT while sacrificing less than 1% in accuracy. RoBERTa matches BERT's training time almost exactly while consistently exceeding BERT by +1.41 percentage points in accuracy.

Padding Speedup on IMDb

Table 6: Dynamic vs. static padding speedup on IMDb by model.

Model	Static (min)	Dynamic (min)	Speedup
BERT-base	32.82	31.57	1.040×
DistilBERT	16.80	16.30	1.031×
RoBERTa-base	33.10	31.48	1.051×
Average	—	—	**1.041**×

4.3 Emotion: Padding Strategy Comparison Across All Models

Table 7: Emotion results: accuracy, F1-macro, and training time by model and padding strategy (mean ± std across 3 seeds). `max_length`= 128.

Model	Strategy	Accuracy (%)	F1-macro (%)	Time (min)
BERT-base	Static	92.70 ± 0.35	88.07 ± 0.19	8.53 ± 0.03
	Dynamic	92.83 ± 0.29	88.55 ± 0.21	7.63 ± 0.08
DistilBERT-base	Static	92.62 ± 0.20	88.13 ± 0.24	4.81 ± 0.07
	Dynamic	92.75 ± 0.15	88.15 ± 0.05	4.34 ± 0.06
RoBERTa-base	Static	92.92 ± 0.42	88.53 ± 0.55	8.94 ± 0.07
	Dynamic	92.90 ± 0.13	88.54 ± 0.32	7.97 ± 0.08

Table 8: Dynamic vs. static padding speedup on Emotion by model.

Model	Static (min)	Dynamic (min)	Speedup
BERT-base	8.53	7.63	1.118×
DistilBERT	4.81	4.34	1.108×
RoBERTa-base	8.94	7.97	1.122×
Average	—	—	**1.116**×

4.4 GPU Memory and Inference Latency

The CSVs also record peak GPU VRAM usage during training and per-sample inference latency on the IMDb test set (25,000 samples), providing a complete resource profile across all models.

Table 9: GPU peak memory usage (MB) and inference latency (ms/sample) on IMDb (mean across 3 seeds).

Model	Strategy	GPU Peak Memory (MB)	Latency (ms/sample)	Latency Speedup vs. BERT static
BERT-base	Static	3591	18.75	1.00×
	Dynamic	3619	18.02	1.04×
DistilBERT-base	Static	2011	8.93	**2.10**×
	Dynamic	2024	8.15	**2.30**×
RoBERTa-base	Static	3810	27.83	0.67×
	Dynamic	3825	26.37	0.71×

DistilBERT's inference advantage (∼2.1–2.3× faster than BERT) is even more pronounced than its training speedup, making it particularly compelling for production inference environments. RoBERTa incurs higher inference latency than BERT despite similar training time, due to its slightly larger parameter count and modified attention patterns. GPU memory usage is consistent across padding strategies (dynamic padding does not reduce training memory since the model itself is identical), but differs substantially across models: DistilBERT uses ∼44% less VRAM than BERT.

5 Discussion

5.1 Architecture: Efficiency vs. Accuracy

The architecture comparison yields two clear practical conclusions:

1. **DistilBERT** (∼1.94× speedup, −0.89% accuracy): A compelling default for scenarios where training budget or inference throughput is constrained and a sub-1% accuracy trade-off is acceptable.
2. **RoBERTa-base** (same speed as BERT, +1.41% accuracy): The preferred choice when accuracy is the primary concern and compute budget allows, as it provides a free accuracy improvement at no extra training cost.

5.2 Revisiting the Padding Hypothesis

The original hypothesis predicted the largest dynamic padding speedup on IMDb (variable-length texts) and the smallest on Emotion (uniform-length texts). The experiments show the opposite. This is resolved by the padding efficiency ratio η (Equation 1):

Table 10: Padding efficiency analysis explaining observed speedups.

Dataset	$\bar{L}$ **(tokens)**	$L_{\max}$	$\eta = \bar{L}/L_{\max}$	**Observed Speedup**
IMDb	∼279	512	≈ 0.54	1.04×
Emotion	∼25	128	≈ 0.19	1.12×

Figure 5 plots all 6 data points on a speedup-vs.-η scatterplot, confirming the inverse relationship.

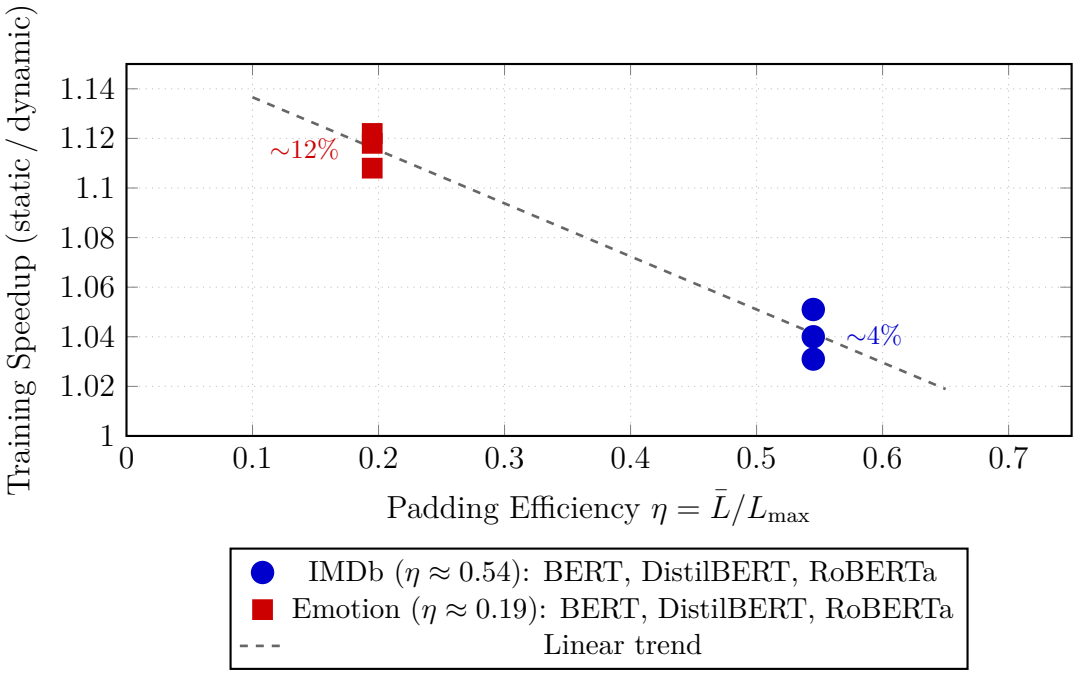

Figure 5: Speedup from dynamic vs. static padding plotted against the padding efficiency ratio η. Each cluster of three points corresponds to one dataset (3 models). Lower η (more wasted padding) yields greater speedup. The dashed line shows the linear best-fit trend.

Revised Hypothesis. *The computational benefit of dynamic padding is determined primarily by the padding efficiency ratio $\eta = \bar{L}/L_{\max}$. When η is low, dynamic padding removes a large proportion of padding tokens and yields large speedups. When η approaches 1, the benefit is negligible. Length variability alone is not a reliable predictor.*

5.3 Practical Guidelines

Table 11: Practical guidelines for padding strategy and model selection.

Condition	Recommendation	Expected Speedup
$\eta < 0.25$	Always use dynamic padding	1.5×–5×
$0.25 \leq \eta < 0.60$	Dynamic padding recommended	1.1×–1.5×
$\eta \geq 0.60$	Marginal; either strategy	$< 1.1\times$
Accuracy is primary	Use RoBERTa-base	+1.4% over BERT
Speed/budget is primary	Use DistilBERT	1.94× faster
Balanced	Use BERT-base	Baseline

6 Limitations and Future Work

Limitations.

- **Single GPU**: All experiments run on a single Colab Pro T4 GPU. Speedup ratios may differ on other hardware (e.g., A100, H100, multi-GPU) due to different memory bandwidth and CUDA core counts.
- **Two datasets**: Results are validated on two datasets representing two distinct regimes ($\eta \approx 0.19$ and $\eta \approx 0.54$). The intermediate regime is not directly covered.
- **Fixed hyperparameters**: Batch size, learning rate, and epoch count were held constant. Dynamic padding benefits may interact non-trivially with batch size.
- **Small seed set**: Three seeds provide statistical robustness but are insufficient for rigorous hypothesis testing; p-values are not reported.

Future Work.

- Vary `max_length` (128, 256, 384, 512) on the same dataset to map the full speedup-vs.-η curve with finer granularity.
- Extend to larger architectures: DeBERTa-v3, ALBERT, and decoder-only models (GPT-2, LLaMA).
- Combine dynamic padding with complementary efficiency techniques (quantisation, gradient checkpointing, flash attention).
- Investigate sequence-to-sequence models (T5, BART) where encoder and decoder padding interact.

7 Conclusion

This study provides a controlled, multi-seed evaluation of architecture selection and data preprocessing strategy in transformer fine-tuning, extended to three model families (BERT, DistilBERT, RoBERTa) across two datasets with contrasting text-length profiles.

The principal conclusions are:

1. **DistilBERT** delivers a $\sim$1.94$\times$ training speedup over BERT with a -0.89% accuracy cost on IMDb, and a $\sim$2.2$\times$ inference speedup, making it the recommended choice when efficiency is constrained.
2. **RoBERTa-base** matches BERT in training time while consistently outperforming it by $+1.41\%$ in accuracy, making it the preferred architecture when compute budget allows.
3. **Dynamic padding** provides $\sim$4% time reduction on IMDb and $\sim$11–12% on Emotion, with no accuracy penalty in either case and consistent results across all three architectures.
4. The **padding efficiency ratio** $\eta = \bar{L}/L_{\max}$ reliably predicts the benefit of dynamic padding: compute η before deciding on a collation strategy, and use dynamic padding whenever $\eta < 0.60$.
5. All experiments required **Google Colab Pro** (paid tier) due to the session duration and GPU memory demands of training three models across 36 runs; the free tier does not provide sufficient resources for experiments of this scale.

These results offer concrete, empirically validated guidance for fine-tuning pipeline design decisions that arise in practice.

References

[1] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Proceedings of NAACL-HLT 2019.

[2] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter*. NeurIPS EMC^2 Workshop 2019.

[3] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. arXiv preprint arXiv:1907.11692.

[4] Hugging Face. (2024). *Hugging Face LLM Course*. https://huggingface.co/learn/llm-course

[5] Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). *Learning Word Vectors for Sentiment Analysis*. Proceedings of ACL 2011.

[6] Saravia, E., Liu, H.-C. T., Huang, Y.-H., Wu, J., & Chen, Y.-S. (2018). *CARER: Contextualized Affect Representations for Emotion Recognition*. Proceedings of EMNLP 2018.